

计算机视觉课程课后作业三

建智 2201 谢天赐 20221082044

选做题 3.1:

参照已有的代码，即将图片竖直方向移动代码，对其进行修改后得到了按照水平移动的代码，主要功能函数如图 1 所示。其原理就是将图像文件解码后以 numpy 数组形式存储，使用一个无限循环调用移动函数。移动函数原理就是将最右侧的像素转移给最左侧的像素。

```
def move_image_horizontal(img):  
    tmp = img[:, -1, :].copy()  
    img[:, 1:, :] = img[:, :-1, :]  
    img[:, 0, :] = tmp  
    return img
```

图 1

运行效果如图 2 所示。

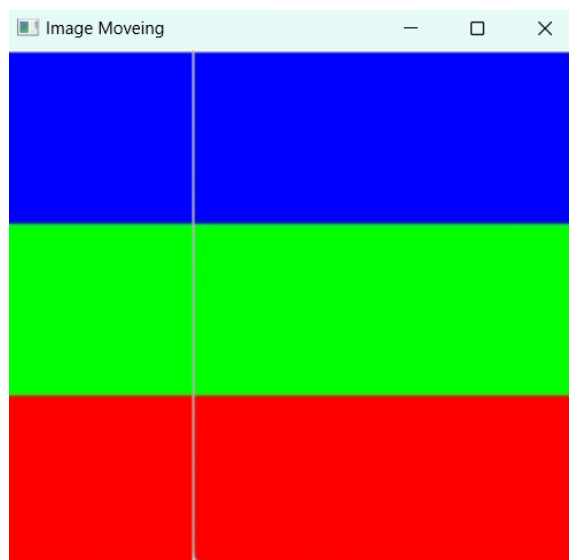


图 2

同理，将最右侧的像素不断转移给最左侧，最下侧的像素不断转移给最上侧，同时进行即可达到斜向移动的效果，主要代码如图 3 所示。

```
def move_image_edges(img):  
    # 保存最下面的一行  
    bottom_row = img[-1, :, :].copy()  
    # 保存最右边的一列  
    right_column = img[:, -1, :].copy()  
  
    # 将除了最下面一行的所有行向上移动一行  
    img[1:, :, :] = img[:-1, :, :]  
    # 将除了最右边一列的所有列向左移动一列  
    img[:, 1:, :] = img[:, :-1, :]  
  
    # 将保存的最下面一行放到最上面  
    img[0, :, :] = bottom_row  
    # 将保存的最右边一列放到最左边  
    img[:, 0, :] = right_column  
  
    return img
```

图 3

运行结果如图 4 所示。



图 4

问题一 3.2:

按照问题要求，得到结果如图 5 所示。

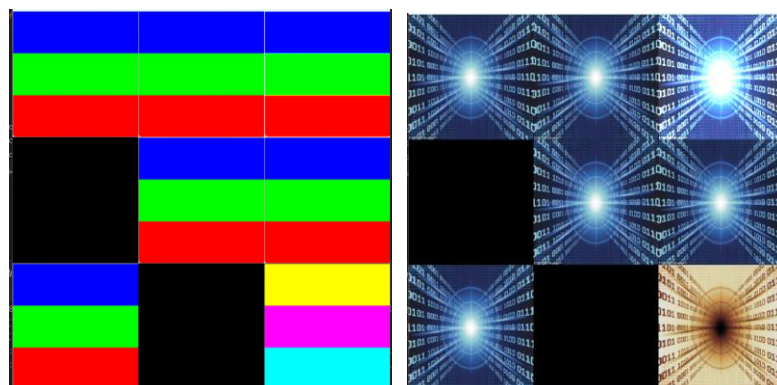


图 5

可以看到成功达到了预期效果，部分源代码如图 6 所示。

```
def ch02_图像算术与逻辑运算():
    fn1 = tk.filedialog.askopenfilename(title="请选择第一张图像文件", filetypes=[('Image files', '*.jpg *.png *.bmp'), ('All files', '*')])
    fn2 = tk.filedialog.askopenfilename(title="请选择第二张图像文件", filetypes=[('Image files', '*.jpg *.png *.bmp'), ('All files', '*')])
    flags = cv2.IMREAD_COLOR
    img1 = cv2.imread(fn1, flags)
    img2 = cv2.imread(fn2, flags)

    if img1 is None:
        img1 = cv2.imdecode(np.fromfile(fn1, dtype=np.uint8), -1)
        print("imdecode 转换得到的第一张彩色图: shape=" + str(img1.shape))
    else:
        print("imread 直接读取得到的第一张彩色图: shape=" + str(img1.shape))

    if img2 is None:
```

图 6

- 相加 (cv2.add): 将两张图像的像素值相加。
- 相减 (cv2.absdiff): 计算两图像像素值的绝对差异。
- 加权加法 (cv2.addWeighted): 按给定的权重将两张图像的像素值相加。
- 位与 (cv2.bitwise_and): 对两张图像进行按位与运算。
- 位或 (cv2.bitwise_or): 对两张图像进行按位或运算。
- 位异或 (cv2.bitwise_xor): 对两张图像进行按位异或运算。
- 位非 (cv2.bitwise_not): 对第二张图像进行按位取反运算。

问题二 3.3.1:

按照问题要求，阅读并修改代码。发现以下问题并改正：

- 文件路径灵活性:
 - ◆ 更改后代码尝试从两个可能的路径读取水印图像，增加了程序的灵活性和健壮性。
- 错误处理:
 - ◆ 如果水印图像无法从任一路径读取，更改后代码会打印错误信息并终止执行，这有助于避免后续操作中的错误。
- 代码清晰性:
 - ◆ 更改后代码在处理图像和水印时，步骤清晰，易于理解和维护。
- 图像显示:
 - ◆ 在显示图像时，更改后代码可能更明确地标注了图像的标题，使得最终用户更容易理解每个子图展示的内容。
- 子图布局:
 - ◆ 更改后代码可能更合理地设置了子图的布局参数，使得图像展示更为合适。
- 程序流程控制:
 - ◆ 更改后代码通过适当的条件判断和循环控制，使得程序的流程控制更为清晰。
- 用户交互:
 - ◆ 更改后代码在用户交互方面可能更为友好，例如在文件选择和错误提示方面。

```
def ch02_像素处理应用_水印_嵌入和复原():
    # 文件对话框获得图像文件路径
    fn = tk.filedialog.askopenfilename(title="请选择图像文件", filetypes=[("Bmp files", "*.bmp")])

    # 确保文件路径正确
    if not fn:
        print("未选择文件")
        return

    # 使用此方法读取图像
    img = cv2.imdecode(np.fromfile(fn, dtype=np.uint8), cv2.IMREAD_GRAYSCALE)
    if img is None:
        print("读取图像失败, 请检查文件路径和格式")
        return
```

图 7

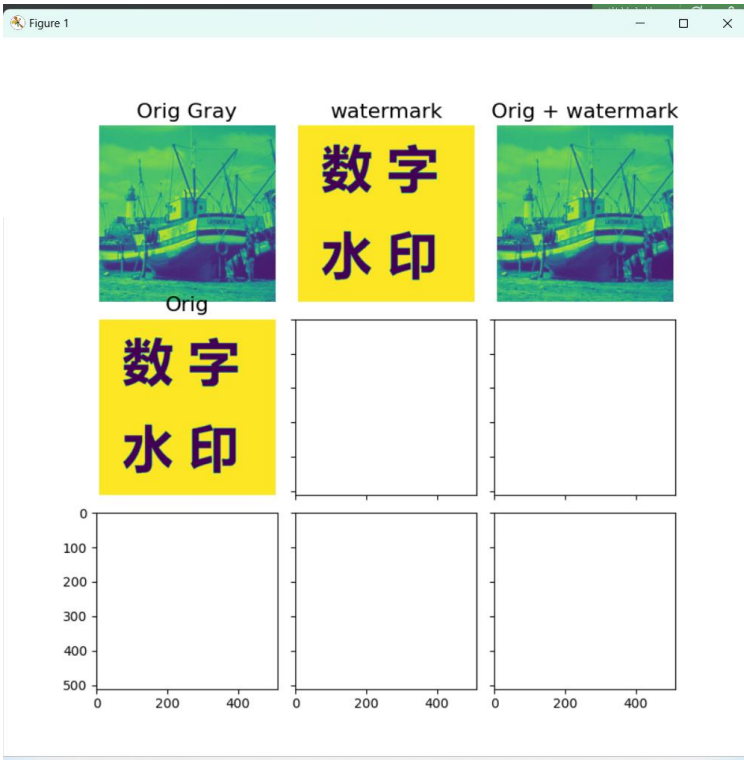


图 8

问题三 3.3.2:

完成代码如图 9 所示:

```

def main():
    root = tk.Tk()
    root.withdraw()
    fn = filedialog.askopenfilename(title="请选择图像文件", filetypes=[("Jpg files", "*.jpg"), ("Bmp files", "*.bmp")])
    if not fn or not os.path.exists(fn):
        print("文件路径错误或文件不存在")
        return
    img = cv2.imread(fn, dtype=np.uint8, -1)
    if img is None:
        print("无法读取图像文件")
        return
    if len(img.shape) == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    watermark = create_watermark_text("XTC20221082044", img.shape[1], img.shape[0])
    embedded_image = embed_watermark(img, watermark)
    fig, axs = plt.subplots(1, 3, figsize=(15, 5))

```

图 9

下面是主要函数以及用处：

1. **创建水印图像** (create_watermark_text 函数)：
 - 这个函数生成了一个空白的黑色图像，然后在图像上添加了白色文字。这个过程涉及到计算文字的大小和位置，以确保文字能够居中显示在水印图像上。
2. **调整图像尺寸** (resize_image 函数)：
 - 如果水印图像和原始图像的尺寸不匹配，这个函数会调整水印图像的尺寸以匹配原始图像。这确保了水印图像可以正确地嵌入到任何尺寸的图像中。
3. **嵌入水印** (embed_watermark 函数)：
 - 这个函数将水印图像嵌入到原始灰度图像中。它通过将原始图像的最低有效位 (LSB) 清零，然后将水印图像的像素值嵌入到这些最低有效位上。这是通过逐像素的位与和位或操作实现的。
4. **提取水印** (extract_watermark 函数)：
 - 这个函数从含水印的图像中提取水印。它通过位与操作提取每个像素的最低有效位，然后通过乘以 255 将提取出的水印值放大到可视化的范围。
5. **位操作**：
 - 嵌入和提取水印的过程中使用了位操作，这是数字图像处理中的一个重要概念。通过操作像素值的二进制表示，可以在不显著改变图像外观的情况下嵌入和恢复信息。

运行结果如图 10 所示，自制水印如图 11 所示。



图 10



图 11

问题四——总结文档：

目的与意义

本课程作业旨在通过实践加深对计算机视觉技术在图像处理中应用的理解。通过图像移动、算术与逻辑运算、水印嵌入与提取等操作，探索了图像处理的多样性和深度。这些技能对于未来在图像编辑、安全、隐身通信等领域的实际应用具有重要意义。

方法与技术

图像移动

水平和垂直移动：通过将图像的最右侧像素移动到最左侧，最下侧像素移动到最上侧，实现了图像的水平 and 斜向移动效果。

循环调用移动函数：使用无限循环调用移动函数，实现了动态的图像移动效果。

图像算术与逻辑运算

算术运算：使用 OpenCV 函数 `cv2.add`（相加）、`cv2.absdiff`（相减）和 `cv2.addWeighted`（加权加法）对两张图像进行算术运算。

逻辑运算：使用 OpenCV 函数 `cv2.bitwise_and`（位与）、`cv2.bitwise_or`（位或）、`cv2.bitwise_xor`（位异或）和 `cv2.bitwise_not`（位非）对图像进行逻辑运算。

水印嵌入与提取

创建水印图像：生成一个黑色背景的图像，并在其上添加白色文字，确保文字居中。

调整图像尺寸：确保水印图像与原始图像尺寸匹配。

嵌入水印：将水印嵌入到原始图像的最低有效位。

提取水印：从含水印图像中提取水印信息。

结果与结论

图像移动

成功实现了图像的水平 and 斜向移动，增强了对图像像素操作的理解。

图像算术与逻辑运算

通过算术和逻辑运算，得到了两张图像相加、相减、加权加法、按位与、按位或、按位异或和按位取反的结果，加深了对图像处理算法的掌握。

水印嵌入与提取

嵌入过程：成功将水印信息嵌入到原始图像中，且嵌入后的图像在视觉上与原图无明显差异。

提取过程：能够从含水印图像中准确提取出水印信息，验证了水印嵌入方法的有效性和可行性。

研究摘要

本课程作业通过一系列图像处理技术，不仅提升了对计算机视觉基础知识的理解，还掌握了图像处理中的高级技术，如水印技术。这些技术在版权保护、安全通信等领域具有广泛的应用前景。通过实际操作，增强了理论与实践相结合的能力，为未来在该领域的进一步学习和探索奠定了坚实的基础。